combination of static text and dynamic content. We can customize the template in received.text.erb; this is the email that's sent to confirm an order:

rails7/depot_q/app/views/order_mailer/received.text.erb

```
Dear <%= @order.name %>

Thank you for your recent order from The Pragmatic Store.

You ordered the following items:

<%= render @order.line_items -%>

We'll send you a separate e-mail when your order ships.
```

The partial template that renders a line item formats a single line with the item quantity and the title. Because we're in a template, all the regular helper methods, such as truncate, are available:

rails7/depot_q/app/views/line_items/_line_item.text.erb

```
<%= sprintf("%2d x %s",
            line_item.quantity,
            truncate(line_item.product.title, length: 50)) %>
```

We now have to go back and fill in the received method in the OrderMailer class:

rails7/depot_qa/app/mailers/order_mailer.rb

```
def received(order)
  @order = order

  mail to: order.email, subject: 'Pragmatic Store Order Confirmation'
end
```

What we did here is add order as an argument to the method-received call, add code to copy the parameter passed into an instance variable, and update the call to mail specifying where to send the email and what subject line to use.

Generating Emails

Now that we have our template set up and our mailer method defined, we can